

12.4 Creating an Index

As we saw in [Section 12.1](#), LlamaIndex provides a very convenient function for converting documents into a searchable index: `VectorStoreIndex.from_documents`. The function merely requires the list of documents as a parameter. LlamaIndex performs some interesting steps in the background, which we'll take a closer look at in this section.

First, LlamaIndex nodes are created from the LlamaIndex documents. For this purpose, a text splitter is used such as the `SentenceSplitter`. In contrast to an “ordinary” text splitter, this one attaches importance to not splitting a sentence into multiple nodes.

To give you a feeling for how the splitting of documents into nodes using the `SentenceSplitter` can look in numbers, we've counted the PDF pages from the 18 documents in our example (with the help of a Linux command that we had created by the Claude AI tool, of course) and compared them with the generated nodes. The 378 PDF pages become 466 LlamaIndex nodes. We've printed parts of the content of such a node without embedding in the form of a JSON structure for you here:

```
{
  "page_label": "5",
  "file_name": "Vielfalter-Falter_Tirol_2018_bis_2022.pdf",
  "file_path": "/data/pdfs/Vielfalter-Falter_Tirol_2018_bis_...",
  "file_type": "application/pdf",
  "file_size": 3190932,
  "creation_date": "2024-07-23",
  "last_modified_date": "2024-07-23",
  "_node_content": "{ \"id\": \"2d677c1e-da77-459d-9282...\",
    \"text\": \"Im Expert*innen-Monitoring werden inner ...\",
    \"class_name\": \"TextNode\" }",
  "_node_type": "TextNode",
  "document_id": "94bdaa9f-c86c-4cf2-b38b-6d03805d265c",
  "doc_id": "94bdaa9f-c86c-4cf2-b38b-6d03805d265c",
  "ref_doc_id": "94bdaa9f-c86c-4cf2-b38b-6d03805d265c"
}
```

You'll recognize some metadata about the document. The content (text) is assigned to the `_node_content` key.

You can control this transformation of documents into nodes via the additional `transformations` parameter when calling `VectorStoreIndex.from_documents`. Then, a list of functions to be processed gets transferred:

```
from llama_index.core import VectorStoreIndex
from llama_index.core.extractors import TitleExtractor
from llama_index.core.node_parser import SentenceSplitter
from llama_index.embeddings.openai import OpenAIEmbedding

index = VectorStoreIndex.from_documents(
    documents=documents,
    transformations=[
        SentenceSplitter(),
        TitleExtractor(),
        OpenAIEmbedding(),
    ],
    show_progress=True,
)
```

After the `SentenceSplitter`, we use the `TitleExtractor`, which is another module from the `LlamaIndex` library. It tries to generate a short summary of the split text with the help of an LLM. (We'll show you how to set the LLM a little later.) The process significantly slows down indexing because the LLM is queried for each of the 466 nodes but also leads to better results in the subsequent search.

Note that LLMs in the cloud usually have limits on how many API calls they can make in a certain period of time. At Anthropic, the API limits for the Claude 3.5 Sonnet model we used were 50 requests and 40,000 tokens per minute. The limit of requests alone is quickly exceeded with the 466 requests that are executed within a few minutes. We therefore used the `TitleExtractor` together with a local Ollama instance, where the limits are only specified by its hardware.

The third entry in the list of `transformations` refers to the embedding model. This is where it gets really exciting because the model has a major influence on the generated index, and the choice of models is almost endless.

12.4.1 The Embedding Model

We use embedding models to convert text passages from the PDF documents and the loaded web pages into vectors. However, a vector doesn't represent a word or a token, but an entire paragraph or a whole page. We'll show you an example of what such a vector can look like. Depending on how large the training data was and how many dimensions the vectors have, embedding models can range in size from a few hundred megabytes to several gigabytes. The derived vectors that we'll save later, on the other hand, only take up a few megabytes for our dataset.

The `OpenAIEmbedding()` call shown previously loads the proprietary `text-embedding-ada-002-v2` model from OpenAI. To use it, you need an API key and credit in your OpenAI account. However, you can also use an open-source model from the Hugging Face platform:

```
from llama_index.embeddings.huggingface import (
    HuggingFaceEmbedding
)

embed_model_hf = HuggingFaceEmbedding(
    cache_folder="/home/user/huggingface_cache",
    model_name="BAAI/bge-m3",
)
```

It makes sense to specify the `cache_folder` for Hugging Face, as otherwise the model has to be reloaded from the internet again and again. Even if embedding models aren't as large as local LLMs, they quickly add up to a few gigabytes.

The `BAAI/bge-m3` used here has been trained with more than 100 languages and is available under the free MIT license. The vectors have 1,024 dimensions, and the model can process inputs with up to 8,192 tokens. However, the memory requirement of over 8 GB shouldn't be underestimated. If you only work with English texts, the much smaller `BAAI/bge-base-en-v1.5` model is a good alternative.

Hugging Face, the Platform for AI Models

At <https://huggingface.co> you'll find a large number of models as well as the *Massive Text Embedding Benchmark* leaderboard (<https://huggingface.co/spaces/mteb/leaderboard>), a ranking of current models with their parameters.

If you're looking for the right model for your purpose, you can filter by topic or keyword here. You'll also find training data for many of the free models.

If you operate a local Ollama server, you can also install embedding models there and use them for index generation:

```
from llama_index.embeddings.ollama import OllamaEmbedding

embed_model_ol = OllamaEmbedding(model_name="mxbai-embed-
large")
```

Here, mxbai-embed-large uses 1,024 dimensions and has a memory requirement of 670 MB. In our tests with PDF documents and websites, the model provided useful results, although not as good hits as the much larger BAAI/bge-m3 from Hugging Face.

But what exactly is the embedding model used for? The input data is compared with the trained model data, and the vectors for the input data are derived from this. The embedding, that is, the vector, for the LlamaIndex node shown previously looks as follows, for example:

```
[ 0.010941985063254, -0.007302129175513, 0.017867475748062, ... ]
```

We've refrained from printing the remaining 765 floats. The open source BAAI/bge-base-en-v1.5 model, which we used in this experiment, uses 768 dimensions.

Creating the index is a computationally intensive operation. Especially if you use local models (in Ollama or from Hugging Face), a powerful graphics processing unit (GPU) will make a noticeable difference. Modern graphics cards from NVIDIA or the processors in modern Apple computers currently provide the best support. The compute time also depends on the model used. Multiple dimensions and larger models require more computing time but also provide better results.

For embedding models from the cloud, you don't need any powerful hardware yourself because that work takes place in the data centers of the selected provider. However, to calculate the vectors, all the data that you index must be uploaded to the cloud. Of course, we don't know what the provider does with your data after the calculation, but we find it highly unlikely that it will simply be deleted.

There are certainly others besides OpenAI from whom you can rent embedding models: Google, MistralAI, CohereAI, IBM, Oracle, Nvidia, and so on—the list reads like a who’s who of the IT industry. Examples of use with the LlamaIndex library can be found at <https://docs.llamaindex.ai/en/stable/examples> under **Embeddings**.

You should also keep in mind that the embedding model isn’t only used to create the index but also for every question you ask. As we’ll see in [Section 12.6](#), the same embedding model that was used to create the index must also be used to convert your questions into vectors.